EDI: A Program to Play the Game of Amazons

Chad Glazier

April 2026

GitHub Repository: https://github.com/Chad-Glazier/edi
Project Documentation: https://chad-glazier.github.io/edi

Contents

1 Introduction **2**

2 Alpha-Beta Search **3**
2.1 The History Heuristic 4
2.1.1 History Updates 4

3 Game-State Evaluation Heuristics **5**
3.1 Territory Evaluation 5
3.2 A Faster Arrow 6

4 Further Work **9**

5 Conclusion **9**

1 Introduction

Highly-optimized and resource-intensive programs have long existed for Chess and other board games. These programs are supported by parallelization, massive transposition tables, and (more recently) neural networks for static evaluation [10, 16]. However, the resource demands for these programs make them ill-suited to many environments—smartphones, web browsers, pay-per-use servers, and so on. Fortunately, other programs have been created with the aim of maximizing performance given far more modest resources [2, 3]. The project presented in this report, named *EDI* ("ee-dee"), takes this approach to the Game of Amazons[1]. *EDI* restricts itself to a single thread and a minimal memory footprint, but has nonetheless proven to be quite competitive.

This document describes and justifies the important algorithms that *EDI* uses, but it does not document the implementation. You can find documentation for the Java version of the project at the link below:

https://chad-glazier.github.io/edi

The documentation at that page also includes a link to the source code, which we have taken efforts to make as readable as possible.

[1] You can read about the game's rules and history on Wikipedia: https://en.wikipedia.org/wiki/Game_of_the_Amazons

2 Alpha-Beta Search

The $\alpha\beta$ ("alpha-beta") pruning optimization for Minimax search [12] is at the heart of many Chess engines, and has been for several decades. More recent research suggests that stochastic models–especially variants of Monte Carlo Tree Search [5]–are superior in games with higher branching factors like Go [17] and Amazons [13]. However, when implementing such a search algorithm for the popular *Invader* Amazons program, Lorentz [8] noted that only carefully optimized and tuned versions of the Monte Carlo program were stronger than the traditional $\alpha\beta$ implementation, and that even those models didn't have perfect records. In short, Monte Carlo appears to be the superior approach, but it might still be beatable. With this in mind, we chose to use $\alpha\beta$ for *EDI* because it's just more fun to root for David than Goliath.

We use the straightforward Negamax implementation of $\alpha\beta$. Our evaluation function always returns a greater value to represent favorability for White and lesser values for positions favorable for Black, so we have to explicitly "color" our scores.

```
function NEGAMAX(state, depth, α, β, color)
    ▷ "color" is -1 for Black and +1 for White. ◁
    ▷ "f" is the leaf node evaluation function. ◁

    if depth = 0 then
        return color · f(state)

    children ← The child states of state
    if size(children) = 0 then
        return color · f(state)

    ▷ "ordering" is the ordering heuristic used, which we discuss elsewhere. Outside of extreme cases,
      only a partial sort is needed. ◁
    children ← children sorted according to ordering.

    best_score ← −∞
    for child ∈ children do
        score ← −NEGAMAX(child, −β, −α, depth − 1, −color)
        best_score ← max{best_score, score}

        if best_score ≥ β then
            break

        α ← max{α, best_score}

    return best_score
```

$\alpha\beta$ search has to use a fixed depth since it cannot feasibly search to the end of the game tree. Choosing a depth is often difficult because the time it takes to complete a search depends on the branching factor and quality of the pruning; factors that cannot be known precisely before the search. Thus, in a time-constrained environment, the typical approach is to begin by searching to a fast, shallow depth, then recording the best move at that depth. Next, the depth is incremented and the search is reiterated. When the time expires, the best move found at the last iteration is returned (i.e., the result of the deepest *completed* search). This process is generally known as *iterative deepening* and it's the approach that *EDI* uses.

2.1 The History Heuristic

It is apparent that the effectiveness of $\alpha\beta$ pruning is heavily reliant on the order in which moves are evaluated. The earlier a cutoff is found, the fewer sub-trees need to be explored. It's typical for $\alpha\beta$ engines to use a *move ordering heuristic* to determine the order in which child nodes are explored. *EDI* exclusively uses the History heuristic which was originally proposed by Jonathan Shaeffer [15].

The History heuristic takes a very intuitive approach; it simply keeps a table of all moves that could be made and increments the score of a move any time that it appears to do well. In our implementation, we use a table which identifies a move based on

- the color of the player making the move,
- the position that the queen starts from,
- the position that the queen moves to, and
- the position where the queen shoots her arrow.

The player color is either White or Black, and there are only 100 unique positions on a 10×10 Amazons board. If the score of a move is represented by a 32-bit integer, then the total size of the table's raw data is $2 \cdot 100^3 \cdot 32$ bits, or about 7.6 MB. For modern computers, this size is trivial.

2.1.1 History Updates

In some programs, including Shaeffer's original version, the history table increments a move's score whenever it produces a cutoff or, if no cutoff was produced, has the greatest score. *EDI* only increments a move's score in the former case.

The decision of an increment function is also important. For *EDI*, we use one known as the *history gravity formula* [4]:

---

function HistoryUpdate(*move*, *depth*)
  ▷ *"M" refers to the maximum history score, which is just a large constant. For EDI, we use* $M = 2^{13}$. ◁
  $bonus \leftarrow \min\{\, depth^2,\, M \,\}$
  $initial \leftarrow history.move$
  $history.move \leftarrow initial + bonus - \lfloor \frac{initial \cdot bonus}{M} \rfloor$

This formula gives larger increments when the cutoff is new (i.e., the initial score was lower), and gives smaller increments when it isn't. This allows newly-found strong moves to be recognized relatively quickly when compared to a fixed increment, while also preventing the oversaturation of established moves. This behavior prevents the table from getting too noisy which is a defect that Shaeffer observed early on with simpler update functions [14].

3 Game-State Evaluation Heuristics

In order to find the minimax score of a move, we need a way to evaluate leaf nodes. Naturally, a stronger evaluation function leads to better performance[2] if both searches reach the same ply in the game tree. However, it has long been supported by empirical evidence that greater depths in game tree search yields better decisions, even when the evaluation function is substantially weaker [1][3]. This leads us to consider whether we should use a very cheap function to maximize search depth, a very strong evaluation function to improve performance directly, or strike some kind of compromise between the two. It's worth noting that, in our experience, we would have to make an evaluation function about ten times faster in order to search one ply deeper.

Different approaches have been taken to handle this tradeoff in Amazons. The stronger programs from the mid-2000s, *Amazong* [7] and *Invader* [6], favor sophisticated evaluation functions which they use for both leaf evaluation and move ordering. In contrast, *Arrow* [9] uses a very simple evaluation in exchange for deeper searches.

3.1 Territory Evaluation

Traditional approaches to static evaluation of Amazons board states have focused on territory evaluation and, to a lesser extent, mobility. At their core, most territory evaluation functions are based on some combination of two terms:

- The number of moves that it would take to move a queen from its current position to each other square if it moved as a chess king does, which we call *king distance*.
- The number of moves that it would take to move a queen from its current position to each other square if it moved as a chess queen does, which we call *queen distance*.

Of course, queens in the game of Amazons move exactly as a chess queen does, so it seems natural to assume that queen distance is a more important term than king distance. However, Lieberum, the creator of *Amazong*, noted that programs which ignore the king distance term tend to let their queens get trapped very often [7]. It's easy to explain this behavior with an example.

Figure 1 represents a common scenario in Amazons where one player is able to isolate their opponent's queen. You can observe that if the territory evaluation is done solely by examining queen distances, then this trap isn't extremely costly for White: the remaining White queen in the region can still reach each square in one or two moves. By contrast, such a trap would be very alarming to a territory evaluation that considers king moves: losing that queen would mean that it can no longer contest almost half of the entire territory. It is very clear to us that the lack of locality in queen distance evaluation creates a glaring vulnerability. The aforementioned *Arrow* program exclusively uses queen distance to partition territory, and we would speculate that this kind of behavior explains the weakness it exhibited in games against more aggressive programs [9].

[2] We use *performance* to refer to how effective the program is; i.e., how much it actually wins. This is not to be confused with the space or time efficiency of a function.

[3] It has also been shown that beyond a certain depth the returns diminish and the quality of the decision can actually worsen if the leaf node evaluation is poor [11]. Fortunately, in Amazons the branching factor is too great for that to be a practical concern.

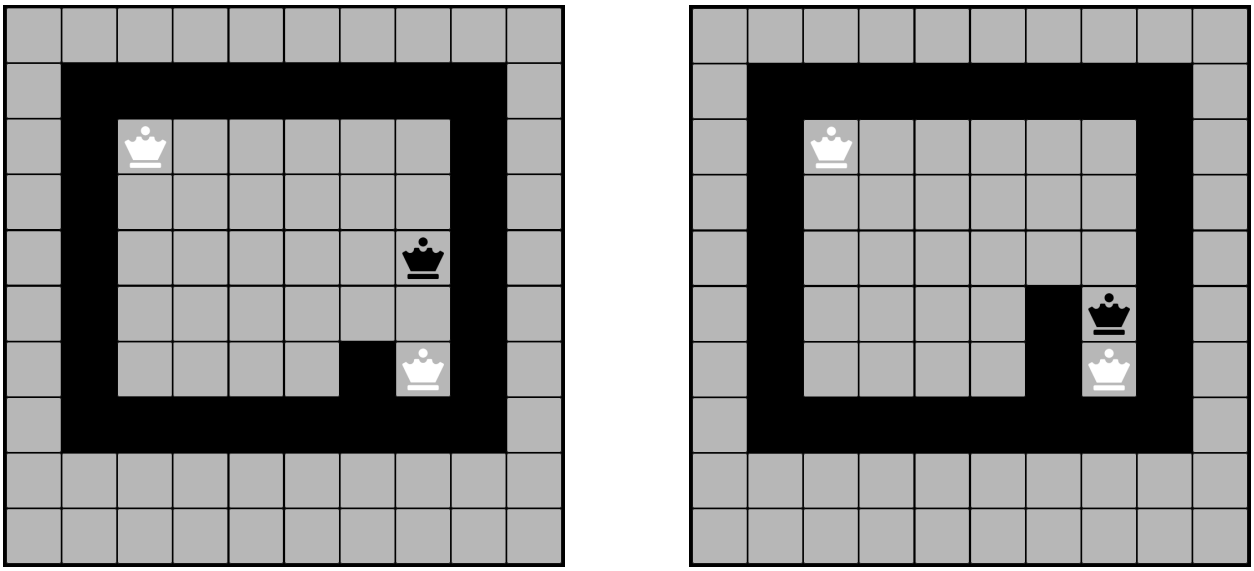

Figure 1: A trap where Black isolates one of the White queens.

3.2 A Faster Arrow

In order to strengthen their evaluation functions, the developers of *Amazong* and *Invader* included a combination of king distance and queen distance evaluations with dynamic weighting functions and nuanced mobility terms. For *EDI*, we take a much more straightforward approach and only consider the king distance on the assumption that it can sufficiently manage mobility and territory control on its own. *EDI*'s leaf node evaluation function is described below.

---

function LeafEvaluation(*state*)
  $score \leftarrow 0$
  for $square \in state.squares$ **do**
    ▷ *"closer" refers to king distance.* ◁
    if *square* is closer to a white queen than every black queen **then**
      $score \leftarrow score + 1$
    if *square* is closer to a black queen than every white queen **then**
      $score \leftarrow score - 1$
  ▷ *"accessible_squares" are ones that can be reached by at least one queen. "|accessible_squares|" refers to the number of such squares.* ◁
  return $\dfrac{score}{|accessible_squares|}$

EDI's function is very similar to *Arrow*'s, except for two points:

1. it partitions territory based on king distance instead of queen distance, and
2. it divides the score by the number of accessible squares.

The reason that we added the division is so that contested squares (i.e., those that are equally distant from

black and white queens) are considered. To explain the importance of this, consider the example where we have two board states s_1 and s_2:

- in s_1, White has 4 squares in its territory, Black has 2, and there are 0 contested squares.
- in s_2, White has 4 squares in its territory, Black has 2, but there are 10 contested squares.

By *Arrow*'s evaluation, these states are equal; both have a score of +2. In reality, these boards are likely to play out very differently: in s_1, Black has to claim territory that is already controlled by White, whereas in s_2 it can instead try to claim more of the contested squares. If we assume that it is easier for Black to claim contested territory than it is to steal territory from White, then s_2 should be considered to be more favorable for Black than s_1. By dividing the territory score by the total number of accessible squares, we can account for such differences. It is also worth noting that this division only takes about two more operations per evaluation with the implementation we use.

The question remains about how much search depth we actually gain by opting for a faster evaluation function. The implementations for the evaluation functions we experimented with are as heavily optimized as they can be (as far as we know); we use bitboards to represent the occupancy, precomputed bitboards for calculating adjacent squares and rays, and directional bitscanning to detect blockers. Figure 2 shows the results of benchmarking the performance of a few different evaluation functions implemented in this way[4]. These benchmarks compare three different evaluation functions:

- The *X+* function is a term from a more recent program [18] which accounts for territory in a manner similar to *Amazong* and *Invader*, though it is simpler and still executes faster than either of them. Its performance is meant to be a generous stand-in for that of the more nuanced functions.
- The *QMinDist* evaluation function is the one that *Arrow* uses.
- The *KMinDist* function is the one that *EDI* uses.

As you can see, *EDI* evaluates board states about 10 times faster than *Arrow*, and about 100 times faster than more sophisticated functions. Based on the rule of thumb we mentioned before, we should expect *EDI* to search about two plies deeper than programs with stronger evaluations. We don't have as any clean, plottable evidence for this claim, but our experience in *EDI*'s tournament matches seems to validate it.

[4]The benchmarks were collected with the industry standard Java Microbenchmark Harness. The benchmarks were executed on a 3.9GHz CPU.

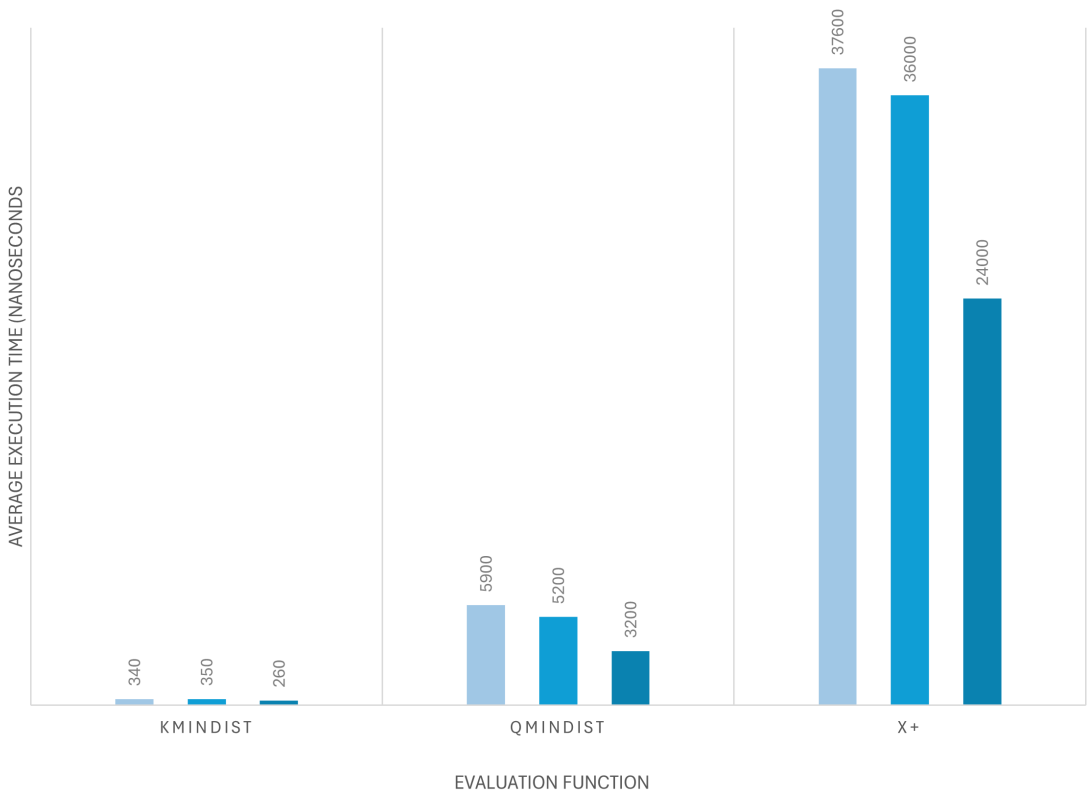

Figure 2: A comparison of the execution speed of various leaf node evaluation functions. For each function, the board was evaluated on different randomized boards with 0 arrows, 10 arrows, and 40 arrows (left to right). As one would expect, boards with more arrows are easier to evaluate because there are less available squares.

.

4 Further Work

In the tournament that *EDI* participated in[5], it performed well and did not lose any of the games it played[6]. However promising this may be, we haven't conducted any real experiments to compare *EDI*'s strength against the existing programs in the literature apart from very limited trials. It might be worthwhile to run a large sample of games between *EDI* and some of the more established programs, including the modern Monte Carlo models, to see how it measures up and whether or not there is any validity to our approach. Since *EDI* is specifically meant to run with constrained resources, such experiments should also include trials with varying restrictions (single-threadedness, memory restrictions, small timers for turns, and so on).

Earlier, we noted a speculative rule of thumb that a tenfold increase to the speed of a static evaluation function allows $\alpha\beta$ to search one ply deeper into the game tree. It would be worthwhile to investigate this further and attempt to find a more precisely-defined relationship in the context of Amazons.

EDI uses its History table as the sole move ordering heuristic. It would be worthwhile to study how effective the History heuristic actually is in Amazons, and whether using different increment functions (and potentially penalizing moves that fail) would improve performance. This approach should also be measured against alternative move ordering heuristics such as applying a strong static evaluation to each child state, an approach that Huerto's report on *Invader* [6] seemed to favor.

5 Conclusion

In this report, we have described the *EDI* program and how it attempts to improve upon existing $\alpha\beta$ programs for Amazons. Our approach can be summarized with the following points:

- *EDI* uses the History heuristic for move ordering, and nothing else, because of it's efficiency and the simple fact that we did not find any other ordering heuristics to be very promising.
- *EDI* uses a minimal evaluation function which partitions territory based on king distance. Apart from being an order of magnitude faster than queen distance functions, we speculate that king distance is sufficient to manage mobility and that it's not worthwhile to compute more nuanced terms because of the impact that slower evaluations have on search depth.

EDI appears to show promise based on its success in the UBC-O tournament, but further work is necessary to determine whether it's actually an improvement over other existing $\alpha\beta$ programs for Amazons.

[5]This tournament was part of an undergraduate AI course at UBC's Okanagan campus; there were 20 groups of four to five members in participation. From speaking to the other groups, we know that there was at least one Monte Carlo program, several *Arrow*-style programs, and a number of other $\alpha\beta$ programs that used more complex evaluation functions.

[6]Our record in the tournament marks that we took four losses in the group stage, but that was because we couldn't connect to the server during the first week and we lost by default. Fortunately, we still went on to take first place.

References

[1] Hans J. Berliner et al. "Measuring the performance potential of chess programs". In: *Artificial Intelligence* 43.1 (1990), pp. 7–20. ISSN: 0004-3702. DOI: https://doi.org/10.1016/0004-3702(90)90067-A. URL: https://www.sciencedirect.com/science/article/pii/000437029090067A.

[2] Chess Programming Wiki. *Ethereal*. Accessed: 2026-03-24. n.d. URL: https://www.chessprogramming.org/Ethereal.

[3] Chess Programming Wiki. *Micro-Max*. Accessed: 2026-03-24. n.d. URL: https://www.chessprogramming.org/Micro-Max.

[4] Chessprogramming Wiki. *History Heuristic*. https://www.chessprogramming.org/History_Heuristic. Accessed: 2026-03-25. n.d.

[5] Rémi Coulom. "Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search". In: *Proceedings of the 5th International Conference on Computers and Games*. CG 2006. Springer, 2006, pp. 72–83. DOI: 10.1007/978-3-540-75538-8_7.

[6] Archie E Huerto. "Experiments in Invader, a Program that Plays Amazons". PhD thesis. California State University, Northridge, 2005.

[7] Jens Lieberum. "An evaluation function for the game of amazons". In: *Theoretical computer science* 349.2 (2005), pp. 230–244.

[8] Richard J Lorentz. "Amazons discover monte-carlo". In: *International Conference on Computers and Games*. Springer. 2008, pp. 13–24.

[9] Martin Müller and Theodore Tegos. "Experiments in computer Amazons". In: *More Games of No Chance* 42 (2002), pp. 243–260.

[10] Yu Nasu. *NNUE: Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi*. In Japanese. Apr. 2018. URL: https://www.apply.computer-shogi.org/wcsc28/appeal/Nasu_Yu.pdf (visited on 02/28/2026).

[11] Dana S. Nau. "QUALITY OF DECISION VERSUS DEPTH OF SEARCH ON GAME TREES". English. Copyright - Database copyright ProQuest LLC; ProQuest does not claim copyright in the individual underlying works; Last updated - 2023-02-24. PhD thesis. 1979, p. 145. ISBN: 9798660711732. URL: https://www.proquest.com/dissertations-theses/quality-decision-versus-depth-search-on-game/docview/302929126/se-2.

[12] Allen Newell, J. C. Shaw, and Herbert A. Simon. "Chess Playing Programs and the Problem of Complexity". In: *IBM Journal of Research and Development* 2.4 (1958), pp. 320–335. DOI: 10.1147/rd.24.0320.

[13] Jianning Quan et al. "Study the performance of search algorithms in Amazons". In: *The 27th Chinese Control and Decision Conference (2015 CCDC)*. IEEE. 2015, pp. 5811–5813.

[14] J. Schaeffer. "The history heuristic and alpha-beta search enhancements in practice". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 11.11 (1989), pp. 1203–1212. DOI: 10.1109/34.42858.

[15] Jonathan Schaeffer. "The history heuristic". In: *ICGA Journal* 6.3 (1983), pp. 16–19.

[16] David Silver et al. "Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm". In: *arXiv preprint arXiv:1712.01815* (Dec. 2017). arXiv: 1712.01815 [cs.AI]. URL: https://arxiv.org/abs/1712.01815.

[17] David Silver et al. "Mastering the Game of Go with Deep Neural Networks and Tree Search". In: *Nature* 529.7587 (Jan. 2016), pp. 484–489. ISSN: 0028-0836. DOI: 10.1038/nature16961.

[18] Runxing Zhong and Ke Zhou. "An Optimization for Amazons Search and Evaluation". In: *2014 Seventh International Symposium on Computational Intelligence and Design*. Vol. 1. IEEE. 2014, pp. 315–318.